

Gemelo digital del beamline

Reporte técnico del proyecto

Taller MIT–UNAL, Medellín

Julio de 2026

Resumen

Este documento describe, de punta a punta, el gemelo digital de un beamline de iones desarrollado sobre SIMION 8.1 para el taller MIT–UNAL. El sistema aprende un modelo surrogate (un *hurdle model*: clasificador de señal + proceso gaussiano físico) a partir de un presupuesto fijo de 100 evaluaciones reales, lo valida contra 394 puntos reales nunca usados en el entrenamiento, y lo usa para resolver dos tareas de control –maximizar la transmisión de iones y alcanzar una consigna objetivo– sin necesitar SIMION en el momento de predecir. El documento cubre tanto la metodología científica (qué se modela, cómo, y con qué evidencia empírica se tomó cada decisión de diseño) como la arquitectura del repositorio de software que la implementa, de modo que sirva como referencia única para entender y reproducir el proyecto completo. Para el informe original entregado en el taller, organizado según las 7 secciones de la rúbrica, ver `report/informe.pdf`.

Índice

1. Introducción y contexto	2
2. Arquitectura del repositorio	2
2.1. Reproducibilidad	3
3. Datos: recolección y partición	4
4. Metodología del modelo	5
4.1. Arquitectura: hurdle model	5
4.2. Feature física: ley V^2 de la lente Einzel	5
4.3. Selección de kernel	5
4.4. Combinando las dos etapas	5
5. Validación	6
5.1. ¿Vale la pena agregar más sondas al entrenamiento?	7
6. Control	7
6.1. El problema: optimizar libremente sobre el gemelo es peligroso	7
6.2. La solución: radio de confianza calibrado con SIMION real	8
6.3. Resultado final verificado	8
7. Diferenciabilidad	9
7.1. Antes y después de la densificación local	9
7.2. El paso ε importa tanto como el modelo	9

8. Decisiones de diseño y alternativas descartadas	10
9. Limitaciones y trabajo futuro	11
9.1. Ruido de SIMION	11
9.2. Hasta dónde confiar en el gemelo	11
9.3. Qué mejoraría con más presupuesto	11
10. Conclusiones	11

1. Introducción y contexto

El reto consiste en dirigir un haz de 500 iones $^{28}\text{Si}^+$ (15 eV) a través de una pila de electrodos electrostáticos hasta un detector, usando SIMION 8.1 como simulador de trayectorias. Ocho electrodos son controlables (lentes Einzel, curvador cuadrupolar y placas deflectoras); el resto quedan fijos (fuente de alto voltaje, detector, tierras). Correr SIMION es costoso –cada evaluación es una simulación completa de trayectorias– así que el taller impone un **presupuesto de entrenamiento de 100 evaluaciones reales** para construir un modelo surrogate (el “gemelo digital”) capaz de:

1. **Predecir** la transmisión (iones que llegan al detector) para cualquier combinación de voltajes, con incertidumbre honesta.
2. **Resolver dos tareas de control** apoyándose solo en el gemelo: encontrar los voltajes que maximizan la transmisión esperada (*dirección*), y encontrar los voltajes que producen una transmisión objetivo dada (*consigna inversa*).
3. **Verificarse** contra corridas reales de SIMION, tanto en predicción puntual como en las recomendaciones de control.

Antes de este proyecto ya existía un historial de **824 evaluaciones reales** generadas por búsqueda bayesiana (Optuna) sobre SIMION (`simulation/optimizer.py`). La estrategia central de este trabajo fue **no desperdiciar ese historial**: en vez de gastar el presupuesto de 100 evaluaciones en puntos nuevos elegidos a ciegas, se seleccionaron 100 puntos de ese historial ya existente por muestreo estratificado, dejando el resto (~ 394 puntos) como conjunto de validación real y gratuito –algo mucho más informativo que cualquier separación sintética.

2. Arquitectura del repositorio

El repositorio separa datos, simulación, código del modelo e informes en carpetas independientes, para que cada pieza pueda evolucionar (y versionarse) por separado:

```
.
+-- src/digital_twin/      El gemelo digital: modelo, control y experimentos.
|   +-- real_data.py       Carga y particion train/test (datos reales).
|   +-- gp_model.py        Hurdle model (clasificador + GP fisico).
|   +-- control.py         Tareas de direccion y consigna inversa.
|   +-- simion_interface.py Conexion a SIMION real.
|   +-- data_collection.py  Utilidades de recoleccion (active learning).
|   +-- main.py            Pipeline completo, de punta a punta.
|   +-- kernel_experiment.py Comparacion de kernels del regresor.
```

```

|   +-- retrain_experiment.py Efecto de agregar mas sondas al training.
|   +-- gradient_eps_study.py Estudio del paso eps del gradiente.
|   +-- gradient_retrain_check.py Chequeo held-out del gradiente.
|   +-- verify_v2.py          Verificacion real de las tareas de control.
|   +-- prototype_synthetic/  Simuladores sinteticos de la etapa de diseno
|                               (no forman parte del gemelo entregado).
+-- data/                      Datos reales de SIMION y caches de reproducibilidad.
|   +-- beamline_results.csv    824 corridas reales (Optuna + SIMION).
|   +-- beamline_study.db      Estudio de Optuna que las genero.
|   +-- gradient_probe_cache.npz 16 sondas de densificacion (congeladas).
|   +-- gradient_eps_study_cache.npz Estudio de paso eps (congelado).
+-- simulation/                Simulacion SIMION (geometria, .iob) y el optimizador.
|   +-- optimizer.py           Script Optuna + SIMION original.
|   +-- SimpleSetUp.iob / .fly2 Definicion del vuelo de iones.
|   +-- electrode_*.stl, electrode_.PA# Geometria de electrodos.
+-- report/                   Informes (LaTeX + PDF) y generador de figuras.
|   +-- informe.tex / .pdf     Informe original (rubrica del taller).
|   +-- reporte_tecnico.tex / .pdf Este documento.
|   +-- generate_figures.py    Regenera las figuras desde datos reales.

```

La Figura 1 resume el flujo de datos de punta a punta.

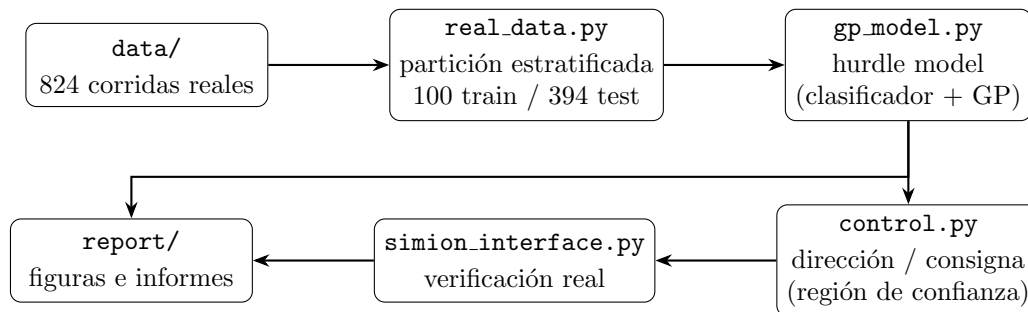


Figura 1: Flujo del pipeline: de los datos reales congelados en **data/** hasta los informes en **report/**, pasando por el ajuste del modelo y las tareas de control. Las flechas de vuelta representan la verificación real (SIMION) y la generación de figuras.

2.1. Reproducibilidad

```

pip install -r requirements.txt
cd src/digital_twin
python main.py

```

Con las semillas fijas (**SEED=42**) y los cachés de **data/** incluidos, esto reproduce exactamente los números de este reporte sin necesitar una instalación local de SIMION. Verificar recomendaciones de control contra SIMION real, o regenerar los cachés de densificación, sí requiere SIMION 8.1 instalado (ver **README.md**).

3. Datos: recolección y partición

De las 824 corridas de `optimizer.py`, 330 fallaron con un error de subproceso. La causa identificada fue interferencia de sincronización de OneDrive sobre el archivo de campo `electrode_PA0` (reescrito en cada evaluación); tras excluir la carpeta de la sincronización, la tasa de error cayó a prácticamente cero. Quedaron **494 evaluaciones reales válidas**: 130 con señal (transmisión > 0) y 364 en cero (zona muerta, 74 % de los puntos).

Del presupuesto de 100 evaluaciones, se eligieron 100 de esos 494 puntos ya existentes mediante *farthest-point sampling* (muestreo de máxima distancia mínima) aplicado por separado a la región de señal y a la zona muerta –45 y 55 puntos respectivamente– para que el pico de transmisión quedara bien representado aunque sea la minoría de los datos. Los 394 puntos restantes, nunca usados para entrenar, forman el conjunto de validación real (Figura 2).

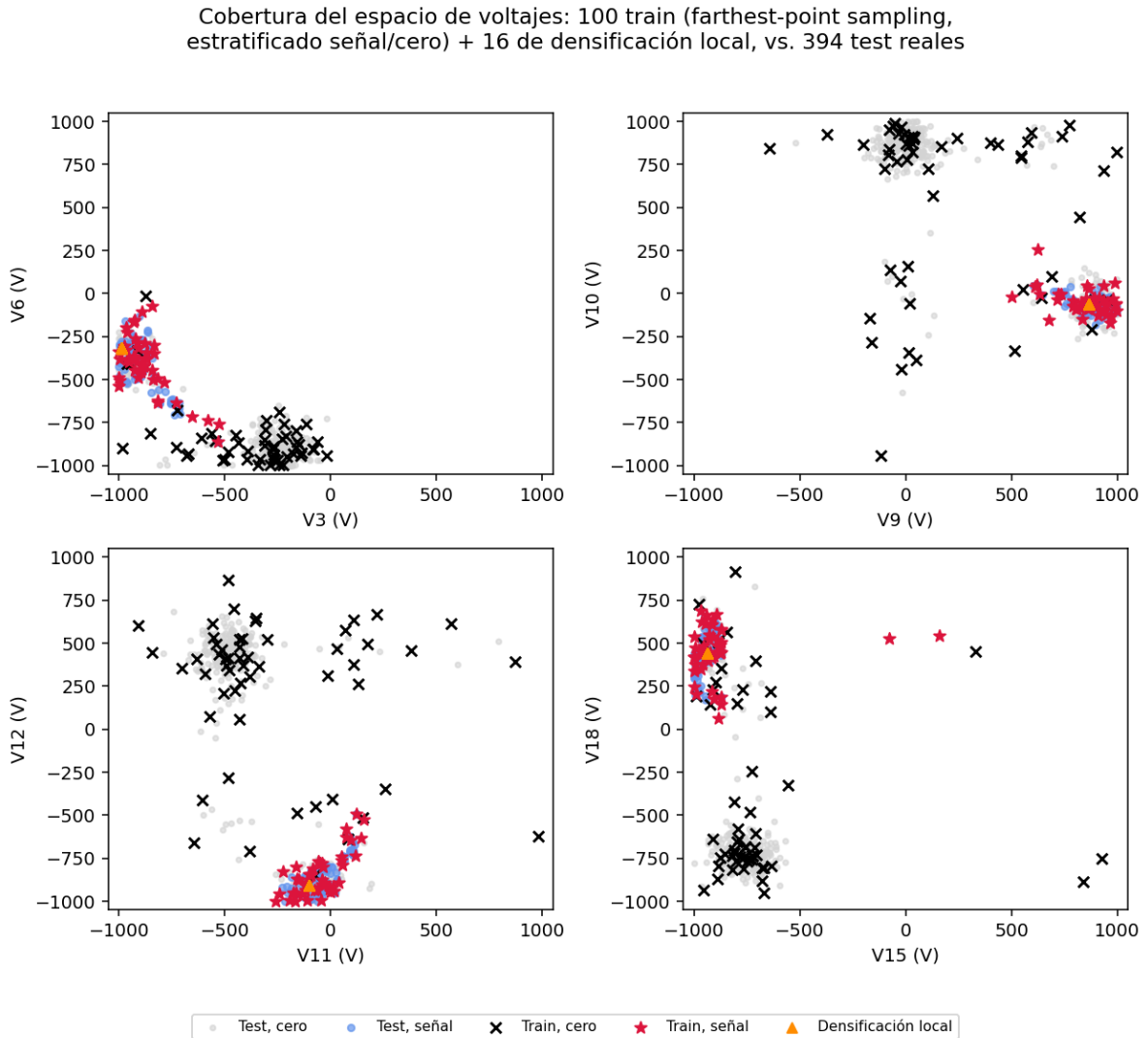


Figura 2: Cobertura del espacio de voltajes en cuatro proyecciones 2D. Los 100 puntos de entrenamiento (cruces y estrellas) se distribuyen por todo el rango explorado, no solo cerca del pico. Los triángulos naranjas son los 16 puntos de densificación local (Sección 7).

Adicionalmente se densificó localmente con 16 puntos reales alrededor del mejor voltaje conocido, cada uno promediado sobre 5 corridas reales de SIMION por el ruido intrínseco de la fuente de iones (Sección 9). En total: **100 + 16 = 116 puntos de entrenamiento**, usando 180 llamadas reales a SIMION en total ($100 + 16 \times 5$).

4. Metodología del modelo

4.1. Arquitectura: hurdle model

Con el 74 % de los puntos válidos en cero, un solo proceso gaussiano ajustando toda la transmisión de una vez predice cerca de cero en todas partes. El problema se separa en dos etapas (*hurdle model*, implementado en `gp_model.BeamlineTwin`):

- **EtapA A – clasificador:** un `GaussianProcessClassifier` (kernel RBF con un length-scale por dimensión, ARD) entrenado con los 116 puntos completos, predice $P(\text{señal} | V)$.
- **EtapA B – regresor de magnitud:** un GP (kernel $\text{ConstantKernel} \times \text{Matérn}_{\nu=1,5}(\text{ARD}) + \text{WhiteKernel}$, con `log1p` en la salida y `StandardScaler` en entradas y salida) entrenado **solo** con los puntos de señal.

Ambas etapas reciben 10 features de entrada: los 8 voltajes más V_3^2 y V_6^2 (Sección 4.2).

4.2. Feature física: ley V^2 de la lente Einzel

Los electrodos V_3 y V_6 son lentes Einzel; su potencia de enfoque escala como $(V_{\text{lente}}/V_{\text{haz}})^2$ (fórmula de lente delgada electrostática), independiente del signo del voltaje. Se agregan V_3^2 y V_6^2 como entradas adicionales para que el GP tenga esa cantidad disponible explícitamente, sin depender de que el kernel la infiera solo de datos escasos.

4.3. Selección de kernel

Se comparó el kernel del regresor de magnitud contra el mismo test real de 394 puntos (`kernel_experiment.py`), sin llamadas nuevas a SIMION:

Kernel	R^2 región informativa	MAE (iones)
RBF	0.203	9.04
Matérn $\nu = 1,5$ (elegido)	0.237	9.01
Matérn $\nu = 2,5$	0.232	9.00

Matérn $\nu = 1,5$ asume una función menos suave que RBF, consistente con un pico de transmisión angosto y una respuesta física que no es infinitamente diferenciable.

4.4. Combinando las dos etapas

Con $T = Z \cdot M$, $Z \sim \text{Bernoulli}(p)$ y M la magnitud (media μ_B , varianza σ_B^2), la ley de varianza total da:

$$\mathbb{E}[T] = p\mu_B, \quad \text{Var}[T] = p(1-p)\mu_B^2 + p\sigma_B^2. \quad (1)$$

Esto es exactamente lo que implementa `BeamlineTwin.predict_combined()`. El primer término de la varianza crece cuando el clasificador no sabe si hay señal; el segundo, cuando el regresor no sabe cuánta. Ambas salidas se recortan a $[0, 500]$.

5. Validación

Sobre los 394 puntos de test reales (nunca vistos en entrenamiento):

Métrica	Global (394 pts)	Región informativa (85 pts con señal)
R^2	0.161	0.237
MAE (iones)	6.81	9.01

El clasificador (Etapa A) obtiene $accuracy=0,845$, $precision=0,582$ y $recall=1,000$: nunca descarta un punto que en realidad tiene señal, a cambio de marcar de más algunos puntos en cero como posible señal – el error preferible, dado que perderse la región de señal es mucho más costoso (Figura 3).

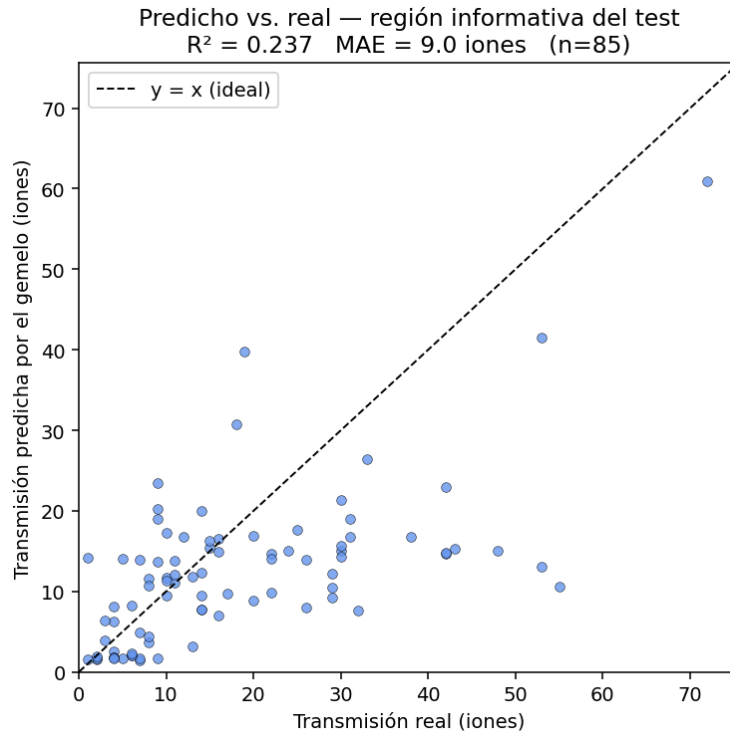


Figura 3: Transmisión predicha vs. real, solo en los 85 puntos de test con señal.

La desviación estándar combinada (Figura 4) es más alta justo donde el clasificador está indeciso ($P(\text{señal}) \in [0,4,0,6]$) y más baja donde está seguro – el comportamiento esperado de una incertidumbre honesta.

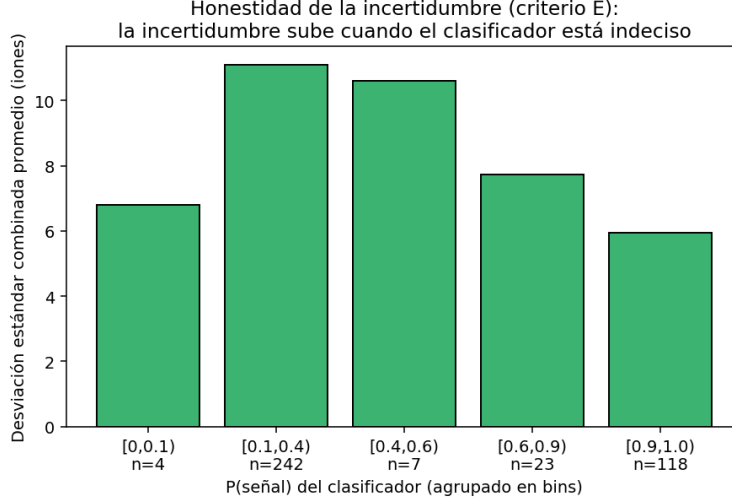


Figura 4: La incertidumbre combinada crece cuando el clasificador está indeciso.

5.1. ¿Vale la pena agregar más sondas al entrenamiento?

Se probó reentrenar agregando las 32 sondas adicionales del estudio de gradiente (Sección 7, $\varepsilon = 5$ y $\varepsilon = 10$ V) al modelo base de 116 puntos (`retrain_experiment.py`):

Puntos de entrenamiento	R^2 región informativa	MAE (iones)
116 (modelo entregado)	0.237	9.01
148 (+32 sondas de gradiente)	0.228	9.12

Agregar esas 32 sondas **degrada** el ajuste global: están apiñadas alrededor de un único punto ancla y desequilibran el GP en el resto del espacio. Por eso el modelo entregado se queda en 116 puntos.¹

6. Control

6.1. El problema: optimizar libremente sobre el gemelo es peligroso

Optimizar `predict_combined` sin restricciones sobre todo ± 1000 V encuentra voltajes donde el modelo está *confiadamente equivocado*: verificado con SIMION real, una recomendación con predicción ~ 35 iones dio **0 iones reales**. Ni penalizar la incertidumbre en el objetivo (*lower confidence bound*) lo arregla – el modelo tiene std bajo justo donde falla. Con solo 45–61 puntos de señal en 8 dimensiones, el pico real es más angosto de lo que el GP puede resolver.

¹El informe original (`informe.pdf`) reporta una caída mayor, a $R^2 = -0.07$, para este mismo experimento. Al reproducir `retrain_experiment.py` con las versiones de `scikit-learn` fijadas en `requirements.txt` se obtiene $R^2 = 0.228$: la dirección de la conclusión (agregar esas sondas degrada el ajuste global) se mantiene, pero la magnitud exacta es sensible a la versión de la librería usada para reoptimizar los hiperparámetros del kernel. Los números de esta sección son los reproducibles con el entorno fijado en este repositorio.

6.2. La solución: radio de confianza calibrado con SIMION real

Ambas tareas de control (`control.py`) restringen la búsqueda (`scipy.optimize.minimize`, L-BFGS-B) a una caja alrededor de un **punto real observado**, con un radio calibrado corriendo SIMION real a distintos valores:

Radio (V)	150	40	20	15	10	5
Transmisión real (iones)	1	0	18	31	29	39

Se fijó **radio** = 5 V. La caja de *dirección* se centra en el mejor voltaje real conocido (72 iones); la de *consigna inversa* se centra en el punto real cuya transmisión observada está más cerca del objetivo, no en el pico – anclar en el pico obligaba al gemelo a modelar la ladera descendente de la campana y dio un error real de 13.8 iones en una versión anterior.

6.3. Resultado final verificado

Cada número real es el promedio de 5 corridas reales de SIMION:

Tarea	Predicho por el gemelo	Real (SIMION, prom. 5)	Resultado
Dirección (max.)	63.3	$55,2 \pm 6,4$	$C = 0,767$ (77 % del óptimo real, 72 iones)
Consigna (obj. =35)	29.9	$36,8 \pm 5,0$	error = 1,8 iones (< ruido de la máquina)

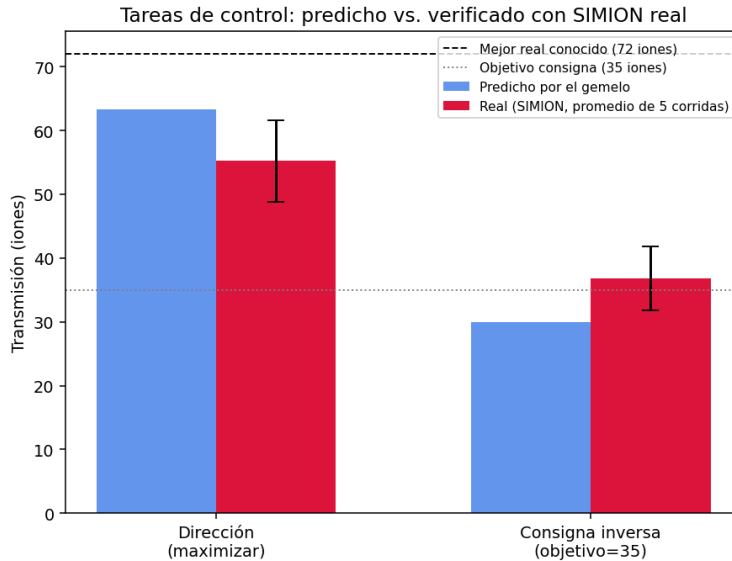


Figura 5: Resultados de control: predicho por el gemelo vs. verificado con SIMION real.

Las funciones `verify_with_simion()` y `verify_gradient_with_simion()` (`control.py`) ejecutan estas verificaciones; no se invocan automáticamente desde `main.py` porque consumen evaluaciones reales adicionales (fuera del presupuesto de entrenamiento, pero reales al fin).

7. Diferenciabilidad

7.1. Antes y después de la densificación local

Se comparó el gradiente del gemelo contra diferencias finitas de SIMION real ($\varepsilon = 1\text{ V}$) en el punto ancla. Antes de densificar (solo 100 puntos base), la similitud coseno entre ambos gradientes era $-0,612$ —¡apuntaban casi en dirección opuesta! Después de densificar con los 16 puntos reales locales, subió a $-0,049$: ya no contradice la realidad, aunque tampoco concuerda bien todavía (Figura 6).

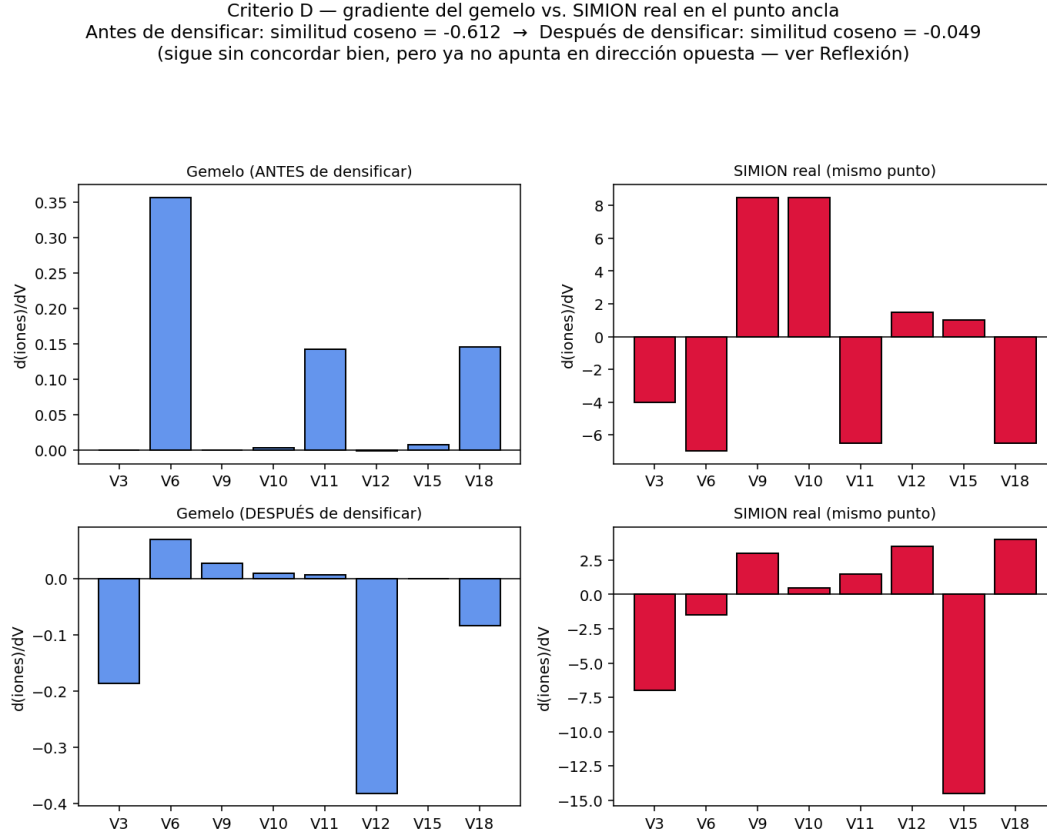


Figura 6: Gradiente del gemelo vs. SIMION real, antes y después de la densificación local.

7.2. El paso ε importa tanto como el modelo

SIMION es ruidoso (Sección 9): incluso promediando 5 corridas, cada conteo conserva $\sim 3\text{--}4$ iones de incertidumbre, y una diferencia centrada con $\varepsilon = 1\text{ V}$ hereda un ruido de $\sim 2,8$ iones/V — mayor que el gradiente verdadero. El “gradiente real” de referencia a $\varepsilon = 1\text{ V}$ era, en buena parte, ruido puro. Repitiendo el chequeo con pasos más grandes (16 puntos $\times$ 5 corridas reales por cada ε , congelados en `data/gradient_eps_study_cache.npz`):

Paso ε	1 V	5 V	10 V
Similitud coseno	$-0,241$	$+0,307$	$+0,059$

Con $\varepsilon = 5\text{ V}$ la relación señal/ruido mejora y el gradiente del gemelo **sí apunta en la dirección correcta**. Con $\varepsilon = 10\text{ V}$ vuelve a caer: el pico real tiene un ancho del orden de $10\text{--}20\text{ V}$, así que la

secante ya se sale de la campana (sesgo de curvatura). Existe una ventana intermedia (~ 5 V) entre el ruido y el sesgo, y es ahí donde el chequeo de diferenciabilidad es informativo (Figura 7).

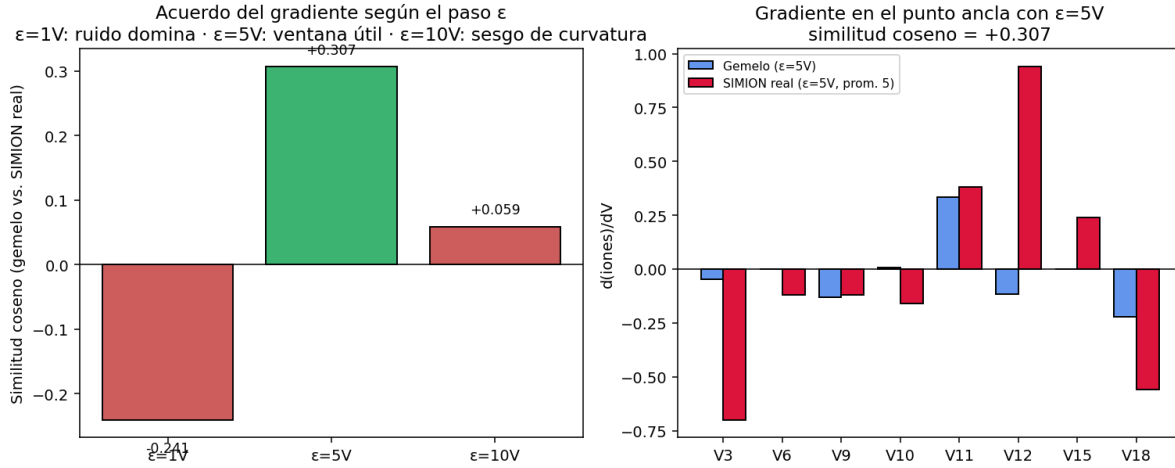


Figura 7: Izquierda: acuerdo del gradiente según el paso ε . Derecha: gradiente del gemelo vs. SIMION real con $\varepsilon = 5$ V, componente por componente.

Como chequeo adicional (`gradient_retrain_check.py`), reentrenar el gemelo agregando las 16 sondas de $\varepsilon = 10$ V sube la similitud a +0,469 contra el gradiente de $\varepsilon = 5$ V como conjunto *held-out* (evaluado sin usarlo en el entrenamiento, para no ser circular). No se incorporó ese reentrenamiento al modelo entregado por la misma razón de la Sección 5.1: mejora el gradiente local a costa del ajuste global.

8. Decisiones de diseño y alternativas descartadas

- **PINNs:** descartadas – no hay una PDE simple que relacione 8 voltajes con el conteo final; esa relación es la integración completa de trayectorias que SIMION ya resuelve.
- **GNN hamiltoniana:** descartada – necesita datos de trayectoria que no se graban (`retain-trajectories=0`) y no hay estructura de grafo natural en 8 voltajes escalares.
- **Regresión simbólica:** se dejó como posible capa de interpretabilidad futura sobre el GP ya entrenado, no como modelo central (riesgosa en 8D con zona muerta).
- **GP simple, sin hurdle ni features físicas** (línea base): R^2 región informativa = $-0,338$, peor que predecir el promedio. Justificó el resto de las decisiones.
- **Hurdle model + features físicas:** subió el R^2 a 0,203; cambiar el kernel a Matérn $\nu = 1,5$ lo llevó a 0,237.
- **Densificar con las 32 sondas de gradiente extra:** probado y descartado (Sección 5.1).
- **Optimización de control sin restricción:** falló catastróficamente (0 iones reales); el radio de confianza calibrado con SIMION real fue necesario.

9. Limitaciones y trabajo futuro

9.1. Ruido de SIMION

El mismo punto exacto de voltajes da resultados distintos en corridas separadas de SIMION (74, 60 y 63 iones observados para un mismo punto): `SimpleSetUp.fly2` muestrea al azar la energía cinética y la dirección de los 500 iones en cada vuelo, y `simion.seed()` solo puede invocarse desde un programa Lua del workbench, no desde `--nogui fly`. La mitigación adoptada en todo el proyecto es promediar 5 corridas reales para cualquier número reportado como verificación (`evaluate_one_real_averaged`).

9.2. Hasta dónde confiar en el gemelo

El gemelo es confiable **dentro de vecindades de ± 5 V de puntos reales conocidos** (los resultados de control lo confirman); **no** para optimización libre sobre todo el rango ± 1000 V, donde puede estar seguro y equivocado a la vez. El $R^2 = 0,237$ en la región informativa, aunque mejor que la línea base, indica que el modelo no ha terminado de aprender la forma fina del pico – esperable con 45–61 puntos de señal en 8 dimensiones.

9.3. Qué mejoraría con más presupuesto

La mejora observada al pasar de 45 a 61 puntos de señal (R^2 de 0,056 a 0,203, y a 0,237 con el kernel final) sugiere que el techo aún no se alcanza. Los puntos nuevos deberían repartirse (p. ej. con `data_collection.expandir_activo()`, muestreo activo guiado por incertidumbre) y no acumularse en el mismo sitio, ya que 32 sondas apiñadas degradan el ajuste global (Sección 5.1). También valdría la pena investigar la fuente residual de fallas de `optimizer.py` (211 de 330 errores no se explican por la interferencia de OneDrive confirmada).

10. Conclusiones

Con un presupuesto de 100 evaluaciones reales (más 16 de densificación local dirigida), el gemelo digital:

- predice la transmisión en la región informativa con $R^2 = 0,237$ sobre 394 puntos de test reales nunca vistos;
- resuelve la tarea de dirección al 77 % del óptimo real conocido, y la consigna inversa con un error de 1.8 iones (menor que el ruido intrínseco de la máquina), siempre que la búsqueda se restrinja a una vecindad de confianza calibrada empíricamente con SIMION real;
- produce un gradiente que, medido a la escala de paso que el ruido de SIMION permite ($\varepsilon = 5$ V), concuerda en dirección con el gradiente real ($\cos = +0,307$).

El principal techo actual es la escasez de datos de señal en 8 dimensiones; el camino más claro para mejorar el gemelo es más presupuesto de entrenamiento repartido activamente cerca de la región de interés, no simplemente más puntos en el mismo sitio.